Ministère de l'Enseignement Supérieur
Université de Manouba
Ecole Supérieure d'Economie Numérique

Support de cours :

Système d'Exploitation

Enseignantes

JIHEN REKIK

Zeineb Dhaoui

Objectif du cours

La partie système s'intéresse aux systèmes d'exploitation modernes et plus particulièrement à

- Définitions générales (architectures et buts)
- Définitions et Historique
- Gestion des processus
- Gestion de la mémoire
- Systèmes de fichiers

Année Universitaire 2012 / 2013

Plan du cours

PLAN DU COURS ---------- II

LISTE DES FIGURES ---------- 1

CHAPITRE 1 INTRODUCTION ---------- 2
I. INTRODUCTION ---------- 2
1. Définition ---------- 2
2. Qualité d'un système d'exploitation ---------- 2
II. HISTORIQUE DES SE ---------- 3
1. Systèmes de traitement par lots *(batch)* simples ---------- 3
2. Systèmes de traitement par lots multiprogrammés en 1965 ---------- 3
3. Systèmes à temps partagé (1960/1970) ---------- 4
4. Systèmes des ordinateurs personnels en 1980 ---------- 4
5. Les systèmes répartis se développent durant les années 80 ---------- 4
III. LES PRINCIPAUX SE ---------- 4
IV. LE ROLE DU SE ---------- 6
1. Le noyau (en anglais Kernel) ---------- 6
2. L'interpréteur de commandes (en anglais Shell) ---------- 7
3. Le système de fichiers (en anglais filesystem) ---------- 7
4. Les Entrées/ Sorties ---------- 7
V. ARCHITECTURE MATERIELLE ---------- 7
1. Le Processeur ---------- 7
2. La mémoire ---------- 7
3. Les interfaces Entrée/sortie ---------- 8
4. Les Bus ---------- 8
CHAPITRE 2 LA GESTION DES PROCESSUS ---------- 9
I. INTRODUCTION ---------- 9
II. LES CONCEPTS DE BASE ---------- 9
III. STRUCTURE D'UN PROCESSUS ---------- 10
1. Structure de l'espace mémoire d'un processus ---------- 10
2. Structure de données pour la gestion des processus ---------- 10
IV. LES ETATS DU PROCESSUS ---------- 11
CHAPITRE 3 ORDONNANCEMENT DES PROCESSUS ---------- 13
I. INTRODUCTION ---------- 13
II. DEFINITION ---------- 13
III. LES ALGORITHMES D'ORDONNANCEMENT SANS REQUISITION ---------- 14
1. Premier arrivé premier servi ---------- 14
2. Le plus court d'abord ---------- 14
3. L'ordonnancement avec priorité simple ---------- 14
IV. ALGORITHMES D'ORDONNANCEMENT AVEC REQUISITION ---------- 15
1. L'algorithme tourniquet ---------- 15
2. L'ordonnancement avec priorité statique multi-niveaux ---------- 15
3. L'ordonnancement avec priorité dynamique multi-niveaux ---------- 16
V. CONCLUSION ---------- 16
CHAPITRE 4 SYNCHRONISATION DES PROCESSUS ---------- 17

I. INTRODUCTION ---- 17
II. NOTIONS DE BASE ---- 17
1. Section Critique ---- 17
2. Exclusion mutuelle ---- 18
III. MECANISMES DE SYNCHRONISATION ---- 18
1. Verrouillage de fichiers ---- 18
2. Solutions avec attente active ---- 19
3. Solutions avec blocage ---- 19
IV. CONCLUSION ---- 20
CHAPITRE 5 GESTION DE LA MEMOIRE ---- 21
I. INTRODUCTION ---- 21
II. ROLE DE LA GESTION DE LA MEMOIRE ---- 21
III. PARTITIONS FIXES ---- 21
IV. PARTITIONS DYNAMIQUES OU VARIABLES ---- 22
V. GESTION AVEC PAGINATION OU SEGMENTATION : LA MEMOIRE VIRTUELLE ---- 22
VI. CONCLUSION ---- 24
CHAPITRE 6 SYSTEME DE GESTION DES FICHIERS ---- 25
I. INTRODUCTION ---- 25
II. LE CONCEPT DE FICHIER ---- 25
III. LES FONCTIONNALITES D'UN SGF ---- 26
IV. ALLOCATION DE L'ESPACE DISQUE ---- 27
1. Allocation contiguë ---- 27
2. Allocation chaînée (non contigüe) ---- 27
3. Allocation non contigüe indexée ---- 28
V. LE SYSTEME DE GESTION DE FICHIERS DE UNIX ---- 29

BIBLIOGRAPHIES ---- 30

Liste des figures

Figure 1-1 Le Système d'exploitation..... 2
Figure 1-2 Système à multiprogrammation..... 4
Figure 1-3 Système à temps partagé 4
Figure 1-4 Structure en couches d'un SE 6
Figure 1-5 Architecture matérielle 7
Figure 2-1 Structure d'un processus 10
Figure 2-2 Changement de contexte..... 11
Figure 2-3 Les états d'un processus..... 12
Figure 3-1 Diagramme de Grantt pour l'algorithme FCFS 14
Figure 3-2 Diagramme de Grantt pour l'algorithme SJF..... 14
Figure 3-3 Stratégie d'ordonnancement Round Robin 15
Figure 3-4 Diagramme de Grantt pour l'algorithme RR avec un quantum trop court 15
Figure 3-5 Diagramme de Grantt pour l'algorithme RR avec un quantum trop long..... 15
Figure 3-6 Ordonnancement avec priorité statique 16
Figure 3-7 Ordonnancement avec priorité dynamique multi-niveaux 16
Figure 4-1 Exemple de processus partageant une même ressource 17
Figure 4-2 Exclusion mutuelle 18
Figure 4-3 Principe de fonctionnement du verrouillage exclusif..... 19
Figure 4-4 Situation d'inter-blocage..... 19
Figure 20 La multiprogrammation 21
Figure 21 Stratégie de partition fixe avec files multiples et file variable..... 22
Figure 22 Stratégie de partition dynamique 22
Figure 23 Gestion avec pagination ou segmentation : la mémoire virtuelle..... 23
Figure 24 Unité de gestion de mémoire (MMU)..... 23
Figure 25 Structure d'un fichier logique..... 25
Figure 26 Structure d'un bloc 25
Figure 27 Exemple de fichier logique 25
Figure 28 Structure d'un répertoire..... 26
Figure 29 Organisation de fichiers et répertoires : Structure arborescente 26
Figure 30 Organisation d'un SGF..... 26
Figure 31 Méthode d'allocation contigüe 27
Figure 32 Méthode d'allocation chaînée..... 28
Figure 33 Organisation des répertoires d'UNIX..... 29

Chapitre 1 Introduction

Ce chapitre a pour objectif de donner un aperçu général des systèmes d'exploitations, définir leurs rôles, leurs compositions ainsi que leurs différents types.

I. Introduction

1. Définition

Système d'exploitation (noté SE ou OS, abréviation du terme anglais Operating System) est :

- Un programme qui agit comme un intermédiaire entre un utilisateur d'un ordinateur et le matériel
- Un ensemble des logiciels effectuant la gestion optimale des ressources d'un système informatique
- Une couche logicielle intercalée entre l'ordinateur et ses applications lancées par ses utilisateurs, lancée au démarrage.

Le système d'exploitation est une couche de logiciel. La **fonction** du système d'exploitation est de masquer la complexité du matériel et de proposer des instructions plus simples à l'utilisateur.

Le système d'exploitation est un gestionnaire de ressources, c'est-à-dire qu'il contrôle l'accès à toutes les ressources de la machine, l'attribution de ces ressources aux différents utilisateurs et la libération de ces ressources lorsqu'elles ne sont plus utilisées. À ce titre, tous les périphériques comme la mémoire, le disque dur ou les imprimantes sont des ressources. Le processeur également est une ressource.

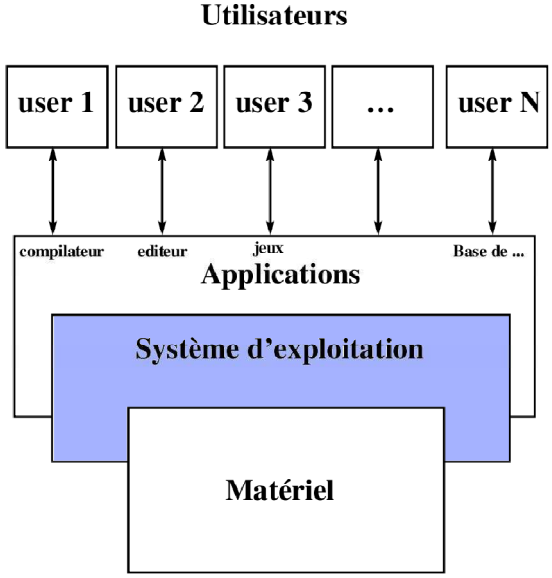


Figure 1-1 Le Système d'exploitation

Qu'est-ce qu'un SE ?

Deux visions :

- Une ***interface*** entre l'utilisateur et le matériel (Cacher les spécificités matérielles à l'utilisateur).
- Un ***gestionnaire de ressources :*** un programme qui gère les ressources de l'ordinateur (processeur, mémoire, périphériques, etc.).
  - ✓ Savoir quelles ressources sont disponibles
  - ✓ Savoir qui utilise quoi, quand, combien, etc.
  - ✓ Allouer/Libérer les ressources efficacement.

2. Qualité d'un système d'exploitation

- **Fiabilité :** Limiter les conséquences des défaillances matérielles ou des erreurs des utilisateurs. En cas de panne, éviter les pertes d'information ou leur incohérence.
- **Efficacité :** Utiliser au mieux les ressources et possibilités matérielles (sans en consommer trop pour lui-même)
- **Facilité d'emploi :** Offrir un langage de commande (dialogue usager-système) et des diagnostics d'erreurs (dialogue système-usager) clairs et précis

- **Adaptabilité :** Permettre les modifications matérielles et logicielles le plus simplement possible à l'aide d'outils spécialisés (processeur de génération de mise à jour)
- **Mesurabilité :** Enregistrer la comptabilité des ressources utilisées par les usagers et mesurer les paramètres de fonctionnement et de charge.

II. Historique des SE

Ordinateur et changements technologiques

- Première génération: Tubes électroniques (lampes à vide)
- Deuxième génération: transistors
- Troisième génération: circuits intégrés
- Quatrième génération: microprocesseurs.
- Cinquième génération: intelligence artificielle.

Partant du fait qu'un système d'exploitation est conçu pour une machine bien particulière, il est logique que les systèmes d'exploitation évoluent avec les générations d'ordinateurs. Avec les progrès de circuit intégré et de processeur, le premier véritable ordinateur dispose d'un véritable système d'exploitation. A l'origine on a commencé par développer du logiciel pour automatiser certaines tâches, pour compiler des programmes et pour réaliser les entrées-sorties. Ce n'est qu'avec la multiprogrammation qu'on a réellement ressenti le besoin d'un système d'exploitation capable de prendre en charge la machine et d'en dispenser le programmeur.
Cette section commence par rappeler les premières réalisations de machines, ensuite nous présentons pour chaque génération d'ordinateurs l'état d'avancement des systèmes d'exploitation.
Evolution historique des SE:

- Le début: routines d'E/S, amorçage système
- Systèmes par lots simples
- Systèmes par lots multiprogrammés
- Systèmes à partage de temps
- Ordinateurs personnels
- SE en réseau
- SE répartis

1. Systèmes de traitement par lots *(batch)* simples

Ce sont les premiers systèmes d'exploitation apparus en 1955.

- L'usager soumet un job (ex: sur cartes perforées) à un opérateur
- L'opérateur place un lot de plusieurs jobs sur le dispositif de lecture
- Un programme, le moniteur, gère l'exécution de chaque programme du lot
- Le moniteur est toujours en mémoire et prêt à être exécuté
- Un seul programme à la fois en mémoire, et les programmes sont exécutés en séquence
- La sortie est normalement sur un fichier, imprimante ou ruban magnétique…

Le principe de fonctionnement des ces systèmes consiste à soumettre un ensemble ou un lot de plusieurs travaux (batch processing) sur le dispositif de lecture de cartes qui seront par la suite exécutés par l'ordinateur. Les programmes sont écrits sur des cartes dites perforées. Un seul programme à la fois est chargé en mémoire et les programmes sont exécutés en séquence.
Un programme est ainsi exécuté et ne sera interrompu que lors d'une opération d'entrée/sortie, d'une erreur ou à la fin de l'exécution.
A la fin, la sortie de chaque travail est envoyé, sur un fichier, imprimante ou ruban magnétique, au programmeur approprié.
Inconvénients :

- Faible utilisation du processeur puisqu'un seul programme est chargé en mémoire.
- Le processeur reste ainsi inactif à chaque opération d'entrée/sortie d'où une perte de temps énorme.

2. Systèmes de traitement par lots multiprogrammés en 1965

Apparition des circuits intégrés.
Les programmes sont transférés vers le disque dès leur arrivée dans la salle machine (spool : Simultaneous Peripheral Operation On Line).
La mémoire est organisée en des partitions contenant au plus un processus. Le système d'exploitation réside aussi dans une partition. Lorsqu'une partition est libre, le système d'exploitation charge un nouveau programme dans la mémoire. La mémoire contenant plusieurs programmes, lorsqu'un processus demande une entrée/sortie, le processeur est alloué à un autre programme en mémoire (c'est ce qu'on appelle la multiprogrammation).

Ceci augmente l'utilisation du processeur puisque les opérations d'E/S sont effectuées de façon asynchrone avec des calculs sur le CPU.
La multiprogrammation nécessite une gestion des interruptions afin de pouvoir exécuter des processus lorsqu'un autre attend une entrée/sortie et un transfert rapide des données sans intervention du processeur.

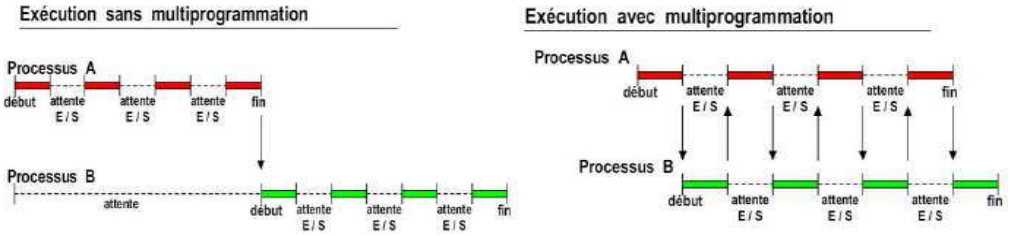


Figure 1-2 Système à multiprogrammation

3. Systèmes à temps partagé (1960/1970)

Le traitement par lots multiprogrammé : une excellente utilisation des ressources mais ne supporte pas l'interaction avec les usagers.
Le système à temps partagé (TSS) permet à la multiprogrammation de desservir plusieurs usagers simultanément.
Les utilisateurs peuvent se connecter à l'ordinateur par l'intermédiaire de leurs terminaux et travailler en même temps.

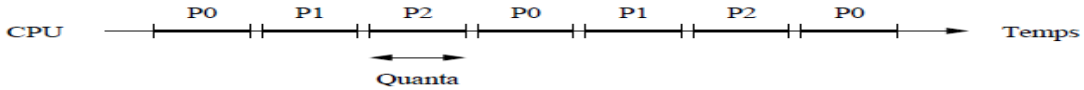


Figure 1-3 Système à temps partagé

Le processeur est alloué, à tour de rôle, pendant un certain temps (quantum) à chacun des programmes utilisateurs en attente d'exécution. A la fin du quantum, l'exécution du programme courant est suspendue et le processeur est alloué à un autre programme. Ce principe donne l'impression aux utilisateurs d'utiliser chacun sa propre machine.
Le processeur est contrôlé par chaque terminal durant la durée du quantum. Lorsqu'un programme est en attente d'une opération d'E/S, il est immédiatement bloqué et le contrôle du processeur est passé à un autre terminal.

4. Systèmes des ordinateurs personnels en 1980

Retour de la monoprogrammation avec les premiers PCs qui mettaient plus l'accent sur la convivialité de l'interface et la rapidité de réaction.
Le besoin de gérer plusieurs applications en même temps conduit au retour de la multiprogrammation.
Plusieurs PCs (clients) peuvent être desservis par un ordinateur plus puissant (serveur) pour des services qui sont trop complexes pour eux. Les grands serveurs utilisent beaucoup des concepts développés pour les systèmes TSS.

5. Les systèmes répartis se développent durant les années 80

Le besoin de gérer plusieurs applications en même temps conduit à redécouvrir la multiprogrammation. Le concept de PC isolé évolue maintenant vers le concept d'ordinateur de réseau *(network computer)*, donc extension des principes des TSS.
Plusieurs PC (clients) peuvent être desservis par un ordinateur plus puissant (serveur) pour des services qui sont trop complexes pour eux (clients/serveurs, bases de données, etc)

- Systèmes d'exploitation répartis: Le SE est exécuté à travers un ensemble de machines qui sont reliées par un réseau
- Systèmes multiprocesseurs
- Systèmes parallèles
- Systèmes à temps réel

III. Les principaux SE

L'évolution des micro-ordinateurs a été encouragée par l'apparition des systèmes d'exploitation adéquats. Les systèmes d'exploitation actuels sont multi utilisateurs. Ils intègrent une gestion des réseaux, et permettent la protection des utilisateurs entre eux. Nous présentons ci-dessous une liste des principaux systèmes sur le marché dans un ordre chronologique (approximatif).

VMS

VMS est la propriété de DEC. Il est apparu avec le premier ordinateur VAX de digital. VMS (Virtual Memory System), est un des premiers systèmes à appliquer la mémoire virtuelle.

CP/M

L'histoire des systèmes d'exploitation pour PC commence avec CP/M créé par Gary Kildall en 1974. A cette époque, chaque ordinateur était livré avec son propre système d'exploitation, étroitement dépendant du matériel utilisé. L'idée de CP/M (Control Program for Microcomputers), une première à l'époque, consistait à créer un système d'exploitation pouvant fonctionner sur les machines de plusieurs constructeurs. C'est d'ailleurs avec CP/M que le Bios apparaît, cette couche logicielle basse permettant d'interfacer matériel et système d'exploitation. CP/M est porté sur presque toutes les plates-formes en vue de l'époque.

MS-DOS

MS-DOS est le plus connu des premiers systèmes d'exploitation pour PC. Ses concepteurs ne se doutaient pas du succès qu'il aurait. Il est mono-utilisateur et mono-tâche. On a du greffer des couches logicielles pour répondre aux évolutions matérielles et aux demandes des utilisateurs. MS-Dos a été rapidement supplanté par les systèmes Windows.

Mac OS

C'est le système d'exploitation de la firme Apple. Il a été livré pour le Macintosh en 1984. C'est un des premiers systèmes à utiliser la souris et une interface graphique avec plusieurs fenêtres.

La version actuelle est la X (prononcer dix). Mac OS X se distingue par un noyau Darwin qui est un open source. Mac OS est un des principaux rivaux des Windows.

OS/2

En 1987, IBM propose le OS/2, plus puissant que le PC avec un nouveau système d'exploitation OS/2 (copropriété d'IBM et de Microsoft). Celui-ci est multitâche. Il est renommé OS/2 Warp Server à partir de la version 3.

NetWare

NetWare de Novell est le premier système d'exploitation réseau 32 bits pour PC. Il intègre un serveur de fichiers et d'impression.

Unix

Unix étant distribué gratuitement à ses tous débuts, il a donné naissance à de nombreuses versions : Les versions les plus connues a ce jour sont Unix SYSTEM V (évolution de la version initiale d'AT&T et Bell) et Unix BSD. Il fonctionne aussi bien sur PC que sur les mini-ordinateurs.

Les principaux Unix du marché sur Intel sont : Open Server et Unixware de SCO (Santa Cruz Operation), Solaris (Sun Microsystems), BSD (Berkeley), Caldera OpenLinux. Cependant trois Unix dominent le monde des serveurs : HP/UX, Sun Solaris, IBM AIX.

Linux

Linux a pris des parts de marché aux Unix, à Novell Netware et à Windows NT-2000 serveur. Il s'est imposé dès la fin du 20ème siècle. Linux est multi utilisateurs, multi tâches, stable et gratuit.

Principales distributions de Linux : RedHat (la plus appréciée des administrateurs de serveurs),

MandrakeSoft (plus facile ou assistée pour débutants), Suze (allemande), Debian, Caldera (devenue payante), Turbolinux (plus connue en Asie).

La famille des Windows :

Microsoft propose en 1992 Windows 3.10 et Windows pour Workgroups 3.11 dont les mots clés sont Multifenêtres et Multitâches coopératif. En 1993, on voit apparaître la première version de Windows NT 3.1 suivie en 1994 par NT 3.5.

L'année 1995, verra la sortie du fort célèbre Windows 95 ("Et soudain le monde devient plus beau"). En 1996, Windows NT 4 avec deux versions station de travail et Serveur. Ensuite, Windows Terminal Server : un système qui simule un environnement multi utilisateurs et prend en charge la connexion de plusieurs terminaux. En 1998 Windows 98.

En 2000, Microsoft commercialise Windows 2000 professionnel et serveur, Windows Millenium, suivi de Windows XP familial et serveur.

Windows 2003 (initialement baptisé .NET) sort en 2003.

Windows XP Octobre 2001. La version comprend entre autres des fonctionnalités réseau et d'administration supplémentaires, est destinée aux entreprises utilisant des réseaux et des serveurs.

Windows Vista contient de nouvelles fonctionnalités (nouvelle interface graphique, une fonction de recherche plus développée grâce à l'indexation, de nouveaux outils de création multimédia, comme le créateur de DVD Windows, la configuration réseau en augmentant le niveau de communication entre les machines sur un réseau local en utilisant la technologie du peer-to-peer qui simplifie le partage de fichiers et de médias numériques entre les ordinateurs et les périphériques).

Windows7 Version commercialisée le 22 Octobre 2009. Divers changements sont au niveau de l'interface et de l'ergonomie générale, un effort particulier pour la gestion transparente des machines mobiles et le souci d'améliorer les performances globales du système (fluidité, rapidité d'exécution même sur des systèmes moins performants, tels les netbooks) par rapport à son prédécesseur.

Windows8 Windows 8 est la dernière version du système d'exploitation. Windows commercialisée depuis le 26 octobre 2012. Windows 8 a été dévoilé, avec l'utilisation de l'interface tactile.

Note Classes de systèmes d'exploitation

- **Mono- tâche (DOS)** *:* A tout instant, un seul programme est exécuté; un autre programme ne démarrera, sauf conditions exceptionnelles, que lorsque le premier sera terminé.
- **Multi- tâches (Windows, Unix, Linux, VMS)** *:* plusieurs processus (i. e. un «programme» en cours d'exécution) peuvent s'exécuter simultanément (systèmes multiprocesseurs) ou en quasi- parallélisme (systèmes à temps partagé)
- **Mono- session (Windows 98,2000)** *:* au plus un utilisateur à la fois sur une machine. Les systèmes réseaux permettent de différencier plusieurs utilisateurs, mais chacun d'eux utilise de manière exclusive la machine (multi- utilisateurs, mono- session)
- **multi- sessions (Windows XP, Unix, Linux, VMS)** *:* Plusieurs utilisateurs peuvent travailler simultanément sur la même machine.

IV. Le rôle du SE

On peut résumer les rôles que joue un système d'exploitation dans un système informatique comme suit :

- Gestion des processus, interruption, ordonnancement, répartition en mémoire, partage des données,
- Gestion du processeur, multiprogrammation, temps partagé, parallélisme
- Gestion de la mémoire principale et de la mémoire secondaire
- Gestion des fichiers
- Gestion des Entrées/Sorties (périphériques) à faible débit ou haut débit
- Lancement des outils pour l'administrateur du système
- Lancement des outils du système (compilateurs, environnement utilisateur,...)
- Gestion des utilisateurs, protection
- Sécurité
- Réseau

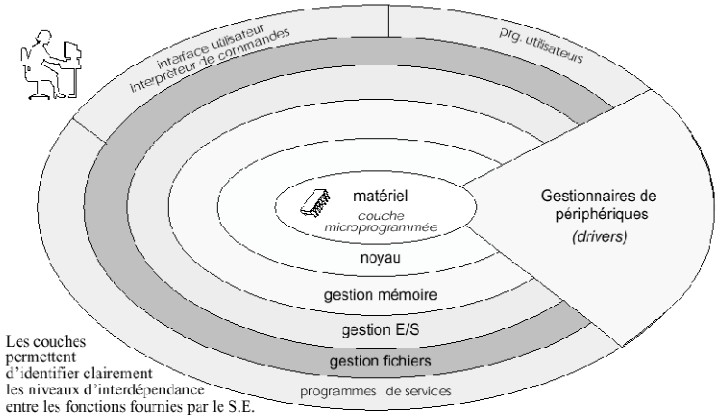


Figure 1-4 Structure en couches d'un SE

1. Le noyau (en anglais Kernel)

C'est la partie la plus importante d'un système d'exploitation. Le noyau d'un système d'exploitation est un ensemble de programmes qui réside constamment en mémoire centrale. Il représente les fonctions fondamentales d'un système d'exploitation telles que la gestion du matériel (mémoire, processeur, périphériques), la gestion des processus (ordonnancement), et les fonctionnalités de communication entre les composants matériels et logiciels d'un ordinateur.

- ✓ Réside en mémoire (fréquence élevée des interventions)
- ✓ Petite taille
- ✓ Gestion du processeur: reposant sur un allocateur (*dispatcher*) responsable de la répartition du temps processeur entre les différents processus, et un planificateur (*scheduler*) déterminant les processus à activer, en fonction du contexte.
- ✓ Gestion des *interruptions* : les *interruptions* sont des signaux envoyés par le matériel, à destination du logiciel, pour signaler un évènement.

- ✓ Gestion du multi- tâches: simuler la simultanéité des processus coopératifs (i. e. les processus devant se synchroniser pour échanger des données) et gérer les accès concurrents aux ressources (fichiers, imprimantes, ...)

2. L'interpréteur de commandes (en anglais Shell)

C'est l'interface entre l'utilisateur et le système d'exploitation. Il permet à l'utilisateur de communiquer avec le système d'exploitation par l'intermédiaire d'un langage de commandes. Ces commandes permettent à l'utilisateur de réaliser des tâches d'administration (ex. useradd person avec Unix) de manipuler des fichiers ou des répertoires (ex. mkdir répertoire avec Unix, md répertoire avec MSDOS), de lancer des applications, de contrôler les périphériques (ex. mount floppy avec Unix) ou d'agir sur la configuration, etc.

3. Le système de fichiers (en anglais filesystem)

C'est une structure de données permettant de stocker des informations et de les organiser dans des fichiers. Il fournit à l'utilisateur une vue abstraite des données enregistrées sous formes d'entités virtuelles. L'utilisateur peut ainsi enregistrer, lire, supprimer et localiser ces entités à partir d'un chemin d'accès.

Le concept de fichiers est une structure adaptée aux mémoires secondaires et auxiliaires permettant de regrouper des données. Le rôle d'un système d'exploitation est de donner corps au concept de fichiers (les gérer, c'est- à- dire les créer, les détruire, les écrire (modifier) et les lires, en offrant la possibilité de les désigner par des noms symboliques).

On peut citer comme exemples de systèmes de fichiers : Fat16, Fat12 de MSDOS, NTFS (New Technology File System) de Windows (introduit en 1994 avec Windows NT), HFS (Hierarchical Filesystem) le système de fichiers standard des ordinateurs Macintosh, ext2-fs Extended Filesystem de linux, etc.

4. Les Entrées/ Sorties

Il s'agit de permettre le dialogue (échange d'informations) avec l'extérieur du système. La tâche est rendue ardue, par la diversité des périphériques d'entrées- sorties et les multiples méthodes de codage des informations (différentes représentations des nombres, des lettres, etc.)

Concrètement, la gestion des E/S implique que le SE mette à disposition de l'utilisateur des procédures standard pour l'émission et la réception des données, et qu'il offre des traitements appropriés aux multiples conditions d'erreurs susceptibles de se produire (plus de papier, erreur de disque, débit trop différent, ...)

V. Architecture matérielle

Un ordinateur est un ensemble de ressources physiques utilisées pour le traitement de l'information.

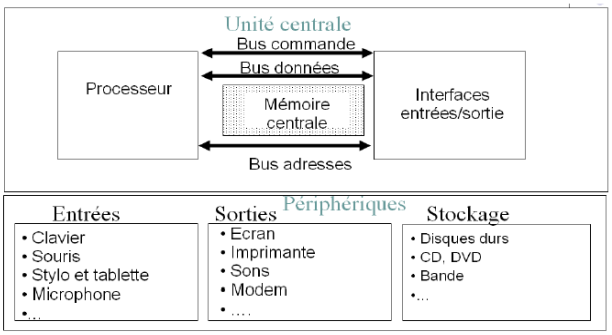


Figure 1-5 Architecture matérielle

Nous donnons dans ce qui suit une définition succincte de ces différentes ressources [Dum05].

1. Le Processeur

Le processeur ou CPU (de l'anglais Central Processing Unit pour Unité centrale de traitement) est le composant responsable de l'exécution des instructions d'un programme chargé en mémoire. Il est capable d'effectuer des opérations arithmétiques et logiques. C'est un circuit électronique cadencé au rythme d'une horloge interne. A chaque top d'horloge le processeur exécute une action, correspondant à une instruction ou une partie d'instruction. La fréquence d'horloge (appelée également cycle, correspondant au nombre d'impulsions par seconde, s'exprime en Hertz (Hz). A chaque top d'horloge le processeur exécute une action, correspondant à une instruction ou une partie d'instruction. Le premier microprocesseur (Intel 4004) a été inventé en 1971 (une unité de calcul de 4 bits, cadencé à 108 kHz).

2. La mémoire

La mémoire centrale appelée également mémoire vive ou interne (RAM = Random Access Memory, mémoire à accès direct) est un espace qui permet de mémoriser temporairement les instructions et les données intermédiaires ou les résultats de calculs du ou des programmes en cours d'exécution. La mémoire vive est volatile, ainsi, à chaque fois que l'ordinateur est éteint, toutes les données présentes en mémoire sont effacées.

La mémoire physique : appelée également mémoire morte ou externe (ROM pour Read Only Memory, mémoire en lecture seule) permet de stocker les informations de manière persistante même lors de l'arrêt de l'ordinateur. La mémoire de masse correspond aux dispositifs de stockage magnétiques, tels que le disque dur, aux dispositifs de stockage optique, correspondant par exemple aux CD-ROM ou aux DVD-ROM.

La mémoire centrale a toujours été une ressource critique: initialement très coûteuse et peu performante (tores magnétiques), elle était de très faible capacité. Pour pallier le manque de mémoire centrale, l'idée est venue d'utiliser des mémoires secondaires (de type disque dur), plus lentes, mais de beaucoup plus grandes capacités.

La mémoire virtuelle repose sur une décorellation entre la mémoire physique (centrale ou secondaire), présente sur la machine, et l'espace mémoire mis à disposition des programmes par le système d'exploitation (la mémoire virtuelle, ou logique).

3. Les interfaces Entrée/sortie

Elles assurent le transfert des données depuis les périphériques d'entrées sorties (capteur, clavier, moniteur ou afficheur, imprimante, modem, etc.) vers la mémoire centrale et vice-versa.

4. Les Bus

Un bus est un ensemble de fils ou câbles qui assurent la transmission du même type d'information. On retrouve trois types de bus : un bus de données bidirectionnel qui assure la circulation des données et des instructions entre le CPU et son environnement ; Un bus d'adresses unidirectionnel qui permet au CPU de sélectionner à chaque instant la case mémoire ou le périphérique auquel il veut faire appel ; Un bus de commande ou bus de contrôle est également géré par le CPU. Il assure la synchronisation des flux d'informations sur les bus des données et le bus des adresses.

Note **Un programme**

Un programme est un ensemble d'instructions et de données. Les instructions d'un programme sont chargées en langage machine (binaire) dans la mémoire afin d'être exécutées dans un ordre bien déterminée par le processeur. Un processus est un programme en cours d'exécution. Les instructions qui constituent un programme peuvent être classifiées en catégories dont les principales sont :

- ✓ Les Instructions d'affectations.
- ✓ Les instructions arithmétiques : telles que les additions, soustractions, divisions ou multiplication.
- ✓ Les instructions logiques : opérations ET, OU, NON, NON exclusif, etc.
- ✓ Les Instructions de branchement (conditionnelle et inconditionnelle)
- ✓ Les Instructions d'entrées/sorties : une instruction d'entrée ou read peut correspondre par exemple à la lecture du disque. Une instruction de sortie ou write peut correspondre par exemple à l'affichage sur l'écran.

L'exécution d'un processus est, en général, une alternance de calculs effectués par le processeur et des opérations d'entrées/sorties effectuées par les périphériques. Il est à noter que le temps consacré aux opérations d'entrées sorties se compte en millisecondes alors que celui consacré aux instructions effectuées par le processeur se compte en nanosecondes. Les opérations d'entrées/sorties sont extrêmement lentes comparées aux autres instructions.

Chapitre 2 La gestion des processus

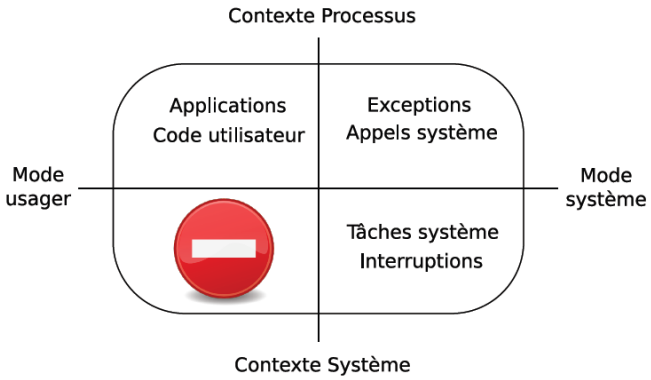
I. Introduction

Un processus (task ou job) est l'entité créée par le système d'exploitation pour l'exécution d'un programme.
Rappel : La **multiprogrammation** : Le processeur se partage entre tous les processus actifs. Ainsi, quand le processeur atteint une instruction d'entrée-sortie, les périphériques interrompt le programme en cours et passe à l'exécution d'un autre programme. La multiprogrammation nécessite le partage du processeur entre les programmes mais également le partage de toutes les ressources disponibles de l'ordinateur (mémoires, périphériques…).
Un programme est stocké sur le disque puis chargé en mémoire afin d'être exécuté. Le processus est l'entité exécutée par le processeur. Le processus est créé par le système d'exploitation ou l'utilisateur au moment où l'exécution du programme doit commencer. Une fois le processus terminé, il est supprimé par le système d'exploitation.
Toutefois un seul programme peut nécessiter plusieurs processus pour son exécution.
La gestion des processus consiste à décider quel processus s'exécute à un moment donné, à quel moment interrompre le processus, quel sera le suivant, et de quelles ressources il a besoin pour son exécution.
Le système d'exploitation manipule deux types de processus : ceux du système et ceux des utilisateurs.
Les fonctionnalités du système d'exploitation en matière de gestion de processus sont :

- La création, suppression et interruption de processus
- L'ordonnancement des processus afin de décider d'un ordre d'exécution équitable entre les utilisateurs tout en privilégiant les processus du système.
- La synchronisation entre les processus ainsi que la communication
- La gestion des conflits d'accès aux ressources partagées
- La protection des processus d'un utilisateur contre les actions d'un autre utilisateur.

II. Les concepts de base

Processus et boot
Le kernel est chargé à partir du disque lors du démarrage du système (Bootstrapping = initialisation du système et définition de l'environnement pour exécuter des processus)
Un premier processus (init) est lancé, s'exécute (réside) jusqu'à l'arrêt du système. Les autres processus sont alors créés à partir d'init (ou d'un de ses descendants).
Processus et espace d'adressage
Plusieurs processus partagent le CPU, la mémoire, les périphériques. Si le système est à temps partagé, l'exécution des processus est virtuellement concurrente. Chaque processus a son espace d'adressage contenant : ses instructions (code) et ses données.
Il contient un ensemble de registres (compteur ordinal, pointeur de pile, ...) nécessaires au bon fonctionnement du processus.
Processus et noyau
La sémantique des interactions entre le kernel et les processus utilisateurs est définie à travers l'API système : les appels système. Ainsi le kernel effectue les instructions pour le compte du processus.
Le kernel intervient également quand un processus effectue une action interdite, division par zéro, débordement de pile (stack over_ow), adressage mémoire incorrect (segmentation fault), cause une exception matérielle (hardware exception) que le kernel traite pour le compte du processus.
Espace d'adressage des processus et noyau
Une partie de l'espace d'adressage de chaque processus correspond à du code et des objets du kernel : c'est la partie espace système. Il n'existe qu'une seule instance du kernel dans le système. Tous les processus pointent sur ce même espace kernel. L'accès à l'espace système se fait en mode système à travers les appels système.
Le kernel gère les processus via 2 objets spécifiques à chaque processus :
- L'espace usager (user space) qui contient des infos sur le processus; Liste des fichiers ouverts; Valeurs des registres, ...
- La pile kernel (kernel stack) pour la gestion des fonctions de tous les processus en appel système.



III. Structure d'un processus

Le système d'exploitation alloue à chaque processus une zone dans la mémoire centrale appelée espace d'adressage du processus. L'espace d'adressage d'un processus comporte une zone de données qui contient les variables globales ou statiques du programme, une zone de code qui correspond aux instructions, en langage d'assemblage, du programme à exécuter et les appels de fonctions, avec leurs paramètres et leurs variables locales, empilées dans une pile d'exécution.

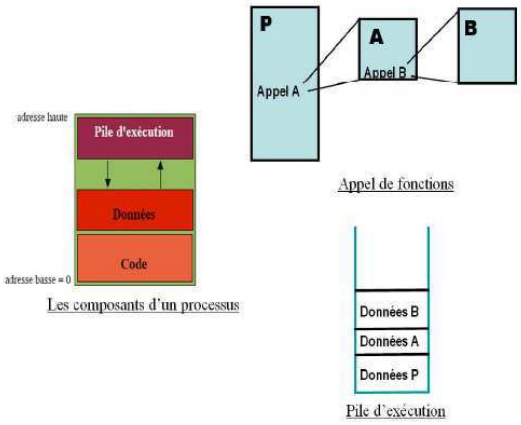


Figure 2-1 Structure d'un processus

L'exécution d'un processus est une alternance entre des calculs effectués par le processeur et des requêtes d'entrée sortie effectuées par les périphériques. Un logiciel est organisé en un certain nombre de processus.

1. Structure de l'espace mémoire d'un processus

- **Le segment de code :** représente le programme à exécuter. Il est toujours placé dans des zones fixes de la mémoire, c'est-à-dire au début de la zone disponible (puisqu'il n'est pas amené à augmenter de taille durant l'exécution du processus).
- **Le segment de données :** Au dessus du segment de code se trouve le segment de données. Ce segment est traditionnellement composé d'un segment de données initialisées (les variables globales et statiques) et d'un segment de données non initialisées qui est créé dynamiquement. Ce segment est amené à croître décroitre suite à l'allocation dynamique de variables.
- **La pile d'exécution :** Pour stocker les données obtenues en cours d'exécution, le système utilise un segment pile. Ce segment est généralement situé en haut de l'espace d'adressage et il croît vers les adresses basses (pile renversée). On utilise souvent les piles lors des appels de fonction. Le système empile le nom de la fonction et les paramètres à passer à la fonction puis les différentes variables locales de la fonction. Une fois la fonction terminée le système dépile toutes les données qu'il a placées pour l'appel de la fonction. Il a à nouveau l'accès aux données qu'il avait stockées avant l'appel de la fonction et peut poursuivre l'exécution du programme.

2. Structure de données pour la gestion des processus

Les systèmes d'exploitation manipulent deux structures de données principales pour gérer les processus créés sur une machine : la table des processus et le bloc de contexte d'un processus.

Contexte d'un processus: appelé aussi bloc de contexte ou de contrôle est une structure de données qui décrit un processus en cours d'exécution. Ce bloc est créé au même moment que le processus et il est mis à jour en

grande partie lors de l'interruption du processus afin de pouvoir reprendre l'exécution du processus ultérieurement. Le contexte d'un processus comporte les informations suivantes :

- ***Le compteur ordinal :*** adresse de la prochaine instruction à exécuter par le processeur
- ***Les contenus des registres généraux :*** ils contiennent les résultats calculés par le processus
- ***Les registres*** qui décrivent l'espace qu'il occupe en mémoire centrale (l'adresse de début et de fin par exemple)
- ***Le registre variable d'état*** qui indique l'état du processus
- D'autres informations telles que la valeur de l'horloge, la priorité du processus,

L'opération qui consiste à sauvegarder le contexte d'un processus et à copier le contexte d'un autre processus dans l'unité centrale s'appelle **changement ou commutation de contexte** (context switching en anglais). La durée d'une commutation de contexte varie d'un constructeur à un autre, elle reste toutefois très faible.

Changement de contexte : Quand le processeur passe de l'exécution d'un processus 0 à l'exécution d'un processus 1, il faut mettre à jour et sauvegarder le PCB de processus 0, reprendre le PCB de processus 1 qui avait été sauvegardé avant et remettre les registres CPU dans la même situation qui est décrite dans le PCB de processus 1 (PCB : Bloc de Contrôle de Processus).

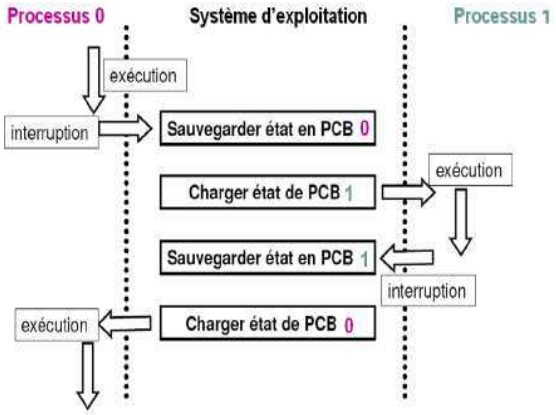


Figure 2-2 Changement de contexte

La table des processus : contient toutes les informations indispensables au système d'exploitation pour assurer une gestion cohérente des processus (informations sur le processus, son exécution, les fichiers qu'il manipule et son occupation mémoire). Elle est stockée dans l'espace mémoire du système d'exploitation, ce qui signifie que les processus ne peuvent pas y accéder.

IV. Les états du processus

Un processus peut passer par plusieurs états avant de finir son exécution. On distingue les quatre états suivants :

- Nouveau : le processus vient d'être créé. Le SE construit un identificateur pour le processus. Les ressources ne sont pas encore allouées.
- Prêt : le processus est prêt et pourra s'exécuter par le CPU dès qu'il sera sélectionné par le système d'exploitation. Toutes les ressources nécessaires lui sont allouées.
- En exécution : le processus est alloué au processeur. Il est entrain d'être exécuté par le CPU.
- En attente : le processus est en attente ou bloqué car il attend des données ou la libération d'une ressource afin de poursuivre son exécution. Pendant ce temps le processeur est attribué à un autre processus.
- Terminé : le processus a terminé son exécution. Un processus peut être terminé par une demande d'arrêt (appel système exit()), à la suite d'une erreur ou bug (division par 0), ou par un arrêt forcé (appel système kill()). Lorsqu'un processus se termine toutes les ressources systèmes qui lui ont été allouées sont libérées par le système d'exploitation.

Les transitions d'un processus

Six transitions peuvent avoir lieu entre ces états :

- ***Nouveau -> Prêt.***
- ***Prêt -> exécution :*** *le processus est choisi (élu) par l'ordonnanceur (scheduleur).*
- ***Exécution -> prêt :*** *résultat d'une interruption causée par un événement indépendant du* processus (ex. Fin du quantum de temps alloué au processus).

- ***Exécution -> attente :*** *lorsqu'un processus fait un appel système (ex. pour une entrée/sortie)* ou lorsqu'il bloqué (ex. attente de la disponibilité d'une ressource partagée).
- ***Attente -> prêt :*** *lorsque l'événement attendu se produit.*
- ***En exécution -> terminé.***

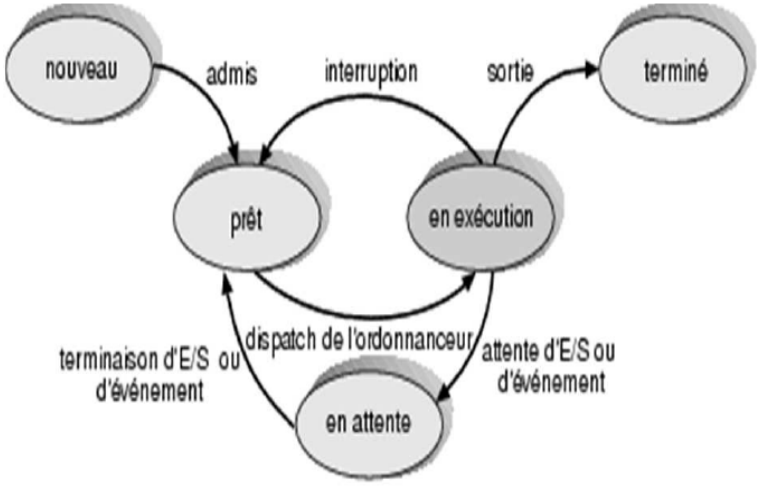


Figure 2-3 Les états d'un processus

Chapitre 3 Ordonnancement des processus

I. Introduction

La multiprogrammation permet à plusieurs processus d'être dans l'état prêt. Un ordinateur possède forcément plusieurs processus en concurrence pour l'obtention du temps processeur. Cette situation se produit lorsque 2 ou plusieurs processus sont en état prêt simultanément. Le processeur est alloué à un seul processus à un moment donné.

Le rôle de l'ordonnanceur (ou scheduler) est de choisir parmi les processus prêts à être exécutés quel processus affecter au processeur. . C'est la partie (un programme) du système d'exploitation responsable de régler les états des processus (Prêt, Actif,…etc.) et de gérer les transitions entre ces états ; c'est l'allocateur du processeur aux différent processus, il alloue le processeur au processus en tête de file des Prêts. Différentes stratégies d'ordonnancement de processus sont utilisées.

Les objectifs d'un Ordonnanceur sont :

- Maximiser l'utilisation du processeur
- Présenter un temps de réponse acceptable
- Respecter l'équité entre les processus selon le critère d'ordonnancement utilisé.

II. Définition

L'ordonnanceur est un programme du noyau qui alloue le processeur aux processus. Son rôle consiste à choisir le processus à exécuter parmi les processus prêts selon une stratégie d'ordonnancement donnée. Les décisions d'ordonnancement peuvent avoir lieu dans l'une des quatre circonstances suivantes :

- Un processus commute de l'état en exécution vers l'état en attente (requête E/S).
- Un processus commute de l'état en exécution vers l'état prêt (quantum de temps utilisé).
- Un processus commute de l'état en attente vers l'état prêt (terminaison d'une E/S).
- Un processus se termine.

Les algorithmes d'ordonnancement de processus sont classés en deux types de stratégie :

- **Ordonnancement coopératif** (sans réquisition) : les processus ne sont pas interrompus, l'ordonnanceur ne choisit un nouveau processus que dans le cas où le processus en cours termine son exécution ou attend une entrée/sortie.
- **Ordonnancement préemptif** (avec réquisition) : l'ordonnanceur peut interrompre un processus en cours d'exécution si un nouveau processus de priorité plus élevée est inséré dans la file des Prêts.

L'objectif d'un algorithme d'ordonnancement consiste à identifier le processus qui conduira à la meilleure performance possible du système. Certes, il s'agit là d'une évaluation subjective dans laquelle entrent en compte différents critères à l'importance relative variable. La politique d'ordonnancement détermine l'importance de chaque critère. Un certain nombre d'algorithmes ont fait leur preuve dans la mise en œuvre d'une politique d'ordonnancement. La liste qui suit passe en revue des critères d'ordonnancement fréquemment utilisés.

- **Utilisation de l'UC :** Pourcentage de temps pendant lequel l'UC exécute un processus. L'importance de ce critère varie généralement en fonction du degré de partage du système.
- **Utilisation répartie :** Pourcentage du temps pendant lequel est utilisé l'ensemble des ressources (outre l'UC, mémoire, périphérique d'E/S…)
- **Débit :** Nombre de processus pouvant être exécutés par le système sur une période de temps donnée.
- **Temps de rotation :** durée moyenne qu'il faut pour qu'un processus s'exécute. Le temps de rotation d'un processus comprend tout le temps que celui-ci passe dans le système. Il est inversement proportionnel au débit.
- **Temps d'attente :** durée moyenne qu'un processus passe à attendre. Mesurer la performance par le temps de rotation présente un inconvénient : Le temps de production du processus accroît le temps de rotation ; Le temps d'attente représente donc une mesure plus précise de la performance. Il faut minimiser le temps d'attente en traitement par lots.
- **Temps de réponse :** Temps moyen qu'il faut au système pour commencer à répondre aux entrées de l'utilisateur. Il faut minimiser le temps de réponse aux processus interactifs.

- **Equité :** degré auquel tous les processus reçoivent une chance égale de s'exécuter. Un bon algorithme d'ordonnancement doit être capable de s'assurer que chaque processus en attente d'exécution reçoit sa part de temps processeur.
- **Priorités :** attribue un traitement préférentiel aux processus dont le niveau de priorité est supérieur.

III. Les algorithmes d'ordonnancement sans réquisition

Avec les algorithmes d'ordonnancement sans réquisition, un processus affecté au processeur ne peut pas être interrompu. La décision d'ordonnancement aura lieu lorsque le processus courant termine son exécution ou réalise une opération d'E/S. Trois algorithmes d'ordonnancement sans réquisition sont présentés dans cette section : premier arrivé premier servi, le plus court d'abord et l'ordonnancement avec priorité simple.

1. Premier arrivé premier servi

L'algorithme d'ordonnancement premier arrivé premier servi (First Come First Served : FCFS) est le plus simple des algorithmes d'ordonnancement. Sa mise en œuvre peut être réalisée grâce à une file d'attente FIFO. Dès que le processeur est libre, il est alloué au processus en tête de la file d'attente. Une fois le processeur est alloué à un processus, celui ci le garde jusqu'à la fin du processus ou suite à la demande d'une entrée/sortie.
Exemple 1. Étant donnés trois processus P1, P2 et P3 dont les durées d'exécution sont données dans le tableau suivant :

Processus	Durée d'exécution
P1	21
P2	3
P3	3

L'exécution de l'algorithme FCFS sur ces processus donne le diagramme de Grantt suivant :

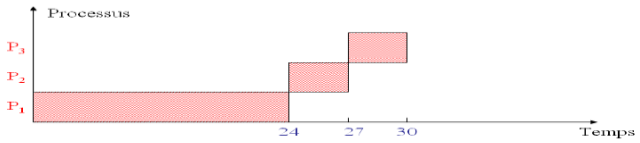


Figure 3-1 Diagramme de Grantt pour l'algorithme FCFS

Le temps d'attente moyen = (0+24+27)/2 = 17ms.
L'algorithme d'ordonnancement FCFS possède l'avantage d'être simple à réaliser. Néanmoins, le temps d'attente pour l'allocation du processeur au processus n'est pas optimal puisque les processus en tête de la file peuvent être les processus les plus longs à exécuter.

2. Le plus court d'abord

L'algorithme d'ordonnancement le plus court d'abord (Shortest Job First : SJF) affecte le processeur au processus possédant le plus court temps d'exécution. Chaque processus est associé à la longueur de son prochain cycle. Le processeur est alloué au processus de plus court cycle.
Dans le cas où plusieurs processus possèdent la même durée, la politique FCFS sera alors utilisée.
Exemple 2 On reprend le même exemple de processus de l'algorithme FCFS, l'exécution de l'algorithme SJF sur ces processus donne le diagramme de Grantt suivant :



Figure 3-2 Diagramme de Grantt pour l'algorithme SJF

Le temps d'attente moyen = (0 + 3 + 6)/3 = 3ms.
Cet algorithme possède l'avantage de donner un temps d'attente optimal. Néanmoins, il est difficile de pouvoir estimer à l'avance le temps d'exécution d'un processus.

3. L'ordonnancement avec priorité simple

Le principe de cet algorithme consiste à attribuer à chaque processus une valeur qui indique sa priorité. Le processeur est alloué au processus de plus haute priorité. La priorité varie selon les systèmes et peut aller de 0 à 127. Un classement possible des processus est donné comme suit (des processus les plus prioritaires aux processus les moins prioritaires) :

1. Processus système.
2. Processus interactifs.
3. Processus batch.

4. Processus utilisateurs.

L'ordonnancement avec priorité sans réquisition peut engendrer un problème de famine puisque les processus de faible priorité risquent d'attendre indéfiniment. Une solution possible à ce problème consiste à faire augmenter la priorité d'un processus avec le temps d'attente. Le processus devient ainsi le plus prioritaire au bout d'un certain temps. Si un "gros" processus utilisateur peu prioritaire devient le plus prioritaire, le temps de réponse aux processus interactifs risque d'augmenter (solution : réquisition).

IV. Algorithmes d'ordonnancement avec réquisition

L'idée de la réquisition est d'interrompre un processus en cours d'exécution à n'importe quel moment. Cette stratégie permet de s'assurer qu'un processus ne prendra pas tout le temps CPU. Une horloge génère une interruption plusieurs fois par seconde. La décision d'ordonnancement peut avoir lieu lorsque le processus courant termine son exécution, réalise une opération d'E/S ou lors de l'interruption d'un processus courant. Trois algorithmes d'ordonnancement avec réquisition sont présentés dans cette section : l'algorithme Tourniquet, l'algorithme avec priorité statique multi-niveaux et l'algorithme avec priorité dynamique multi-niveaux.

1. L'algorithme tourniquet

L'algorithme d'ordonnancement tourniquet (Round Robin : RR) a été conçu pour les systèmes à temps partagé (TSS chapitre 1). Le principe de cet algorithme consiste à définir un intervalle de temps appelé quantum. Le processeur est alloué aux processus à tour de rôle pendant la durée du quantum (figure suivante). Dans la pratique la durée d'un quantum varie entre 10 et 100 ms.

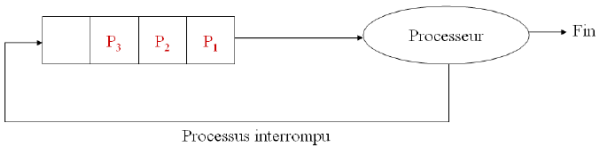


Figure 3-3 Stratégie d'ordonnancement Round Robin

La performance de l'algorithme de Round Robin dépend du choix de la taille du quantum. Un quantum trop court provoque trop de commutations de contexte et abaisse l'efficacité du processeur.

Un quantum trop long augmente le temps d'attente. Il faut trouver un juste équilibre afin d'obtenir un temps d'attente convenable et une perte de temps processeur raisonnable (choisir un quantum ni trop court ni trop long).

Exemple 3 : Quantum de temps trop court. Avec un quantum de temps de 20 ms et une commutation de contexte de 5 ms, le dixième processus attend 9*25= 225 ms. 25% (5/20) du temps est perdu dans la commutation de contexte.

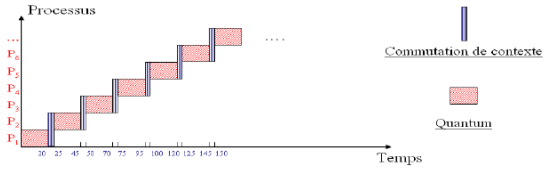


Figure 3-4 Diagramme de Grantt pour l'algorithme RR avec un quantum trop court

Exemple 4 : Quantum de temps trop long. Avec un quantum de temps de 500 ms et une commutation de contexte de 5 ms, le dixième processus attend 9*505= 4,5 s (temps d'attente important). 1% (5/500) du temps seulement est perdu dans la commutation de contexte.

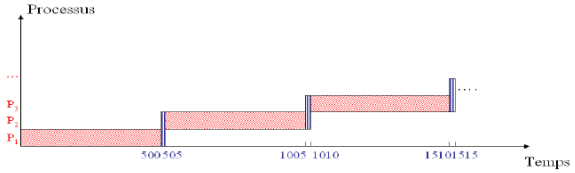


Figure 3-5 Diagramme de Grantt pour l'algorithme RR avec un quantum trop long

2. L'ordonnancement avec priorité statique multi-niveaux

Les processus de même priorité sont regroupés dans une file. Chaque file d'attente est absolument prioritaire par rapport à la file immédiatement inférieure. On peut définir différents algorithmes d'ordonnancement pour chaque file (RR ou FCFS). Par exemple, on utilise l'algorithme d'ordonnancement RR pour la file des processus interactifs (pour diminuer le temps de réponse) et FCFS pour la file des processus utilisateurs.

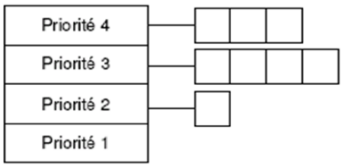


Figure 3-6 Ordonnancement avec priorité statique

Pour qu'un processus d'une file (ou d'un niveau de priorité) puisse s'exécuter, il faut que tous les processus des files plus prioritaires soient vides. Ceci peut engendrer un problème de famine puisque les processus de priorité faible risquent d'attendre indéfiniment. Une solution à ce problème consiste à faire exécuter les processus d'une même file pendant un quantum de temps. A la fin du quantum, le processeur est alloué aux processus de la file suivante. On peut également utiliser l'ordonnancement avec priorité dynamique.

3. L'ordonnancement avec priorité dynamique multi-niveaux

L'ordonnancement avec priorité dynamique multi-niveaux (Multi-level-feedback round robin queues) permet aux processus de changer en permanence de file d'attente. Chaque file d'attente dispose d'un quantum de temps pour exécuter ses processus. A la fin de ce quantum, les processus qui n'ont pas été traités changent de file. Ainsi, un processus est affecté à une file pendant un quantum de temps. Si son exécution n'est pas terminée à la fin du quantum, il passe au queue de la file immédiatement inférieure. Le système d'ordonnancement des processus sous UNIX (BSD 4.3 et system V4) utilise cette stratégie avec pour chaque file d'attente un algorithme d'ordonnancement Tourniquet.

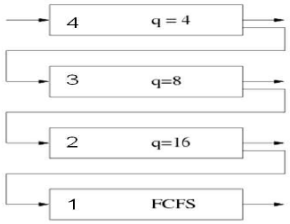


Figure 3-7 Ordonnancement avec priorité dynamique multi-niveaux

Note : Les performances d'un algorithme pour un ensemble de processus donné peuvent être analysées si les informations appropriées relatives aux processus sont fournies. Par exemple, des données sur l'arrivée du processus et sur l'heure d'exécution de ce processus sont nécessaires pour évaluer l'algorithme SRT.

$$Temps\ de\ rotation = Temps\ de\ fin\ d'exécution - Tempsd'arrivée$$

$$Temps\ d'attente = Temps\ de\ rotation - Durée\ d'exécution$$

$$Temps\ moyen\ d'attente = \frac{\sum Temps\ attente}{nbre\ de\ processus}$$

$$Rendement = \frac{\sum Temps\ d'exécution}{nbre\ de\ processus}$$

V. Conclusion

Nous avons présenté dans ce chapitre un composant principal du noyau qui est l'ordonnanceur. Nous avons présenté les algorithmes d'ordonnancement avec leurs avantages et leurs inconvénients. Plusieurs critères sont utilisés pour le choix d'un algorithme d'ordonnancement. Dans la pratique, ces critères ne peuvent pas être tous respectés en même temps. Des critères peuvent être avantagés par rapport à d'autres selon les situations.

Chapitre 4 Synchronisation des processus

I. Introduction

Lorsqu'on observe un écran, il apparaît clairement que l'ordinateur mène plusieurs activités en parallèle : une horloge, par exemple, affiche l'heure pendant qu'on utilise un traitement de texte. Mieux encore on apprécie de pouvoir imprimer un document tout en continuant à écrire un autre texte. Ces activités sont gérées, en apparence, simultanément alors que la machine ne dispose que d'un processeur unique.
Ces processus échangent des informations: ainsi le traitement de texte déclenche régulièrement une copie sur disque qui sauvegarde le texte écrit. Ce logiciel consulte donc l'horloge afin de déclencher ce mécanisme au moment voulu.

Dans un système d'exploitation multiprogrammé en temps partagé, plusieurs processus s'exécutent partagent des objets (mémoires, imprimantes, etc.). Le partage d'objets sans précaution particulière peut conduire à des résultats imprévisibles. D'où la nécessité de définir des règles et des outils pour assurer la bonne synchronisation de l'ensemble. Le but de ce chapitre est définir le problème de synchronisation de processus et de présenter quelques mécanismes de synchronisation proposés.

II. Notions de base

Pour mieux expliquer l'utilité de la synchronisation de processus, nous prenons comme exemple de processus, ceux utilisés pour débiter et créditer un compte bancaire. Soient deux processus P_1 et P_2:

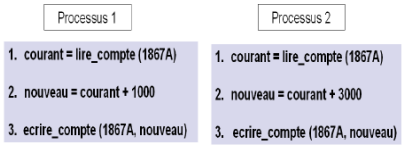


Figure 4-1 Exemple de processus partageant une même ressource

Les deux processus ont accès au même compte bancaire 1867A qui possède comme valeur initiale 1000. Avec la multiprogrammation, l'exécution des opérations des processus P_1 et P_2 peut être entrelacée dans un ordre quelconque, à condition de respecter l'ordre local pour chacun des processus. Ainsi, on peut obtenir les deux exemples d'exécution suivants :
1. Exécution 1 : P_2:1 ; P_2:2 ; P_2:3 ; P_1:1 ; P_1:2 ; P_1:3.
2. Exécution 2 : P_2:1 ; P_1:1 ; P_1:2 ; P_1:3 ; P_2:2 ; P_2:3.
Avec l'exécution 1, la valeur du compte 1867A est égale à 5000. L'exécution 2 donne comme valeur pour le compte 1867A 4000. Il y a ainsi problème avec la deuxième exécution où les opérations des deux processus se sont entrelacées. Pour éviter ce problème, on peut interdire l'entrelacement des opérations qui accèdent à une ressource partagée : le compte 1867A dans l'exemple. L'ensemble de ces opérations qui accèdent à une ressource partagée forme ce qu'on appelle une **section critique**. Le fait d'interdire à deux processus d'entrer en même temps dans leur section critique est ce qu'on appelle l'**exclusion mutuelle**. L'exclusion mutuelle doit être réalisée sans **inter-blocage**. Il faut éviter ainsi aux processus d'attendre indéfiniment l'accès à une section critique. Ces différentes notions sont définies dans ce qui suit.

1. Section Critique

Une section critique est la partie de programme dans laquelle se font des accès à une ressource partagée. Il s'agit d'un ensemble d'instructions qui peuvent produire des résultats imprévisibles lorsqu'elles sont exécutées simultanément par des processus différents. La définition d'une section critique se fait par l'ajout de code de contrôle avant et après la suite d'instructions concernées par le partage de variables ou ressources : un code entrée dans la section critique et un code sortie de la section critique :



Ce code de contrôle est défini par le programmeur. Il doit assurer ce qu'on appelle une exclusion mutuelle. Selon la stratégie de synchronisation utilisée, on peut avoir recours à un code de contrôle différent.

2. Exclusion mutuelle

L'exclusion mutuelle consiste à garantir qu'un seul processus au plus entre dans sa section critique à un moment donné. Ainsi, l'exécution d'une section critique d'une ressource par un processus exclut tous les autres processus d'accéder à leurs sections critiques pour la même ressource.

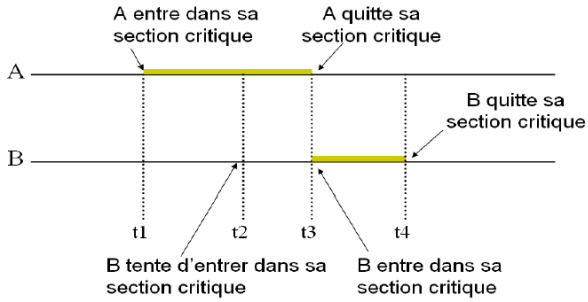


Figure 4-2 Exclusion mutuelle

Les mécanismes de synchronisation qui assurent une exclusion mutuelle doivent vérifier les points suivants :

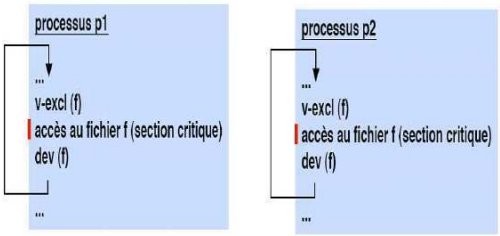
- Un seul processus en section critique : au plus un processus exécute sa section critique à un moment donnée.
- Absence d'inter-blocage : si deux ou plusieurs processus essayent d'entrer dans leurs sections critiques, au moins un réussira. L'inter-blocage est une situation dans laquelle plusieurs processus (au moins 2) sont bloqués. Chacun d'eux ne pouvant être réveillé que par une action de l'un des autres. On ne peut pas sortir d'une situation d'inter-blocage sans intervention extérieure.
- Absence de délai : si un processus P essaye d'entrer en section critique et les autres processus exécutent leur sections non critiques ou sont terminés, alors le processus P ne sera pas retardé d'entrer dans sa section critique.
- Absence de famine : un processus qui essaye d'entrer dans sa section critique va forcement y rentrer.

III. Mécanismes de synchronisation

Différents mécanismes sont proposés pour la synchronisation entre des processus concurrents. Nous présentons dans ce qui suit un aperçu de solutions proposées pour assurer une exclusion mutuelle entre processus concurrents : le verrouillage de fichiers, les solutions avec attente active et les solutions avec blocage.

1. Verrouillage de fichiers

Le verrouillage de fichiers est un des mécanismes de synchronisation les plus utilisés les plus simples à définir. Ce mécanisme est utilisé dans le cas de partage de fichiers. Ainsi, si deux processus P_1 et P_2 utilisent le même fichier f, le début et la fin d'une section critique sont respectivement définis par le verrouillage exclusif et le déverrouillage :



Le verrouillage exclusif garantie les points suivants :

- Un fichier f verrouillé en accès exclusif par un processus ne peut pas être verrouillé par un autre processus. Un processus qui tente de verrouiller un fichier déjà verrouillé en accès exclusif est bloqué (mis en attente du verrou).
- L'opération de déverrouillage réveille un processus en attente du verrou (et un seul). Le verrouillage exclusif garantit comme le montre la figure suivante qu'un seul processus peut entrer dans sa section critique à un moment donné.

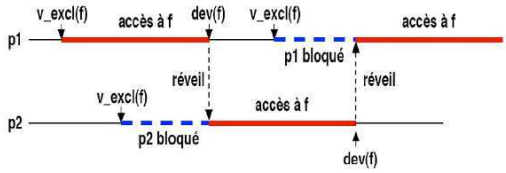
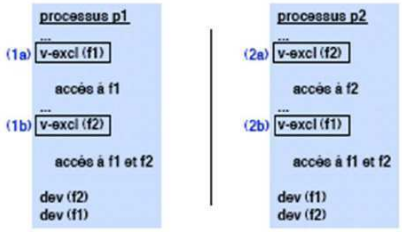


Figure 4-3 Principe de fonctionnement du verrouillage exclusif

Le problème que pose le verrouillage exclusif consiste au fait qu'on peut avoir des situations d'inter-blocage (deadlock). Prenons l'exemple des processus p1 et p2 définis comme suit :



En faisant fonctionner les deux processus, les deux processus p1 et p2 se trouvent dans une situation d'inter-blocage après le deuxième v_excl(f1). Les deux processus p1 et p2 sont bloqués, et le resteront indéfiniment : pour réveiller p1, il faut faire dev(f2) qui ne peut être fait que par p2, bloqué. Pour réveiller p2, il faut faire dev(f1) qui ne peut être fait que par p1 qui se trouve déjà bloqué.

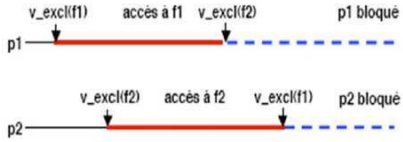


Figure 4-4 Situation d'inter-blocage

Il est nécessaire de noter que le verrouillage exclusif n'est pas toujours nécessaire. Supposons que p1 et p2 lisent le fichier f (sans le modifier), et que p3 écrive dans le fichier. Alors on peut permettre l'accès simultané à f par p1 et p2 et non par p1 et p3 ou p2 et p3. On utilise dans ce cas ce qu'on appelle le verrouillage partagé. Ainsi, un fichier f verrouillé en accès partagé par un processus ne peut pas être verrouillé par un autre processus en mode exclusif, mais peut être verrouillé en mode partagé.

2. Solutions avec attente active

Comme son nom l'indique, dans une solution avec attente active, un processus désirant entrer en section critique attend de façon active qu'une condition soit vérifiée. Cette condition consiste à vérifier qu'aucun autre processus ne se trouve dans sa section critique. Le début d'une section critique est définie par une boucle où la condition de la boucle correspond à la vérification de la disponibilité ou non de la section critique :

```
Tant que (NON condition) Faire
sleep(délai);
Fintantque
<section critique>
```

Un exemple de solution avec attente active est l'utilisation de verrou. Le principe de cette solution consiste à utiliser une variable partagée, le verrou. Cette variable est initialisée à 0 pour indiquer qu'il n y a pas de processus dans la section critique. Pour un processus donné, le début d'une section critique est défini par une boucle (attente active). Cette boucle vérifie si la valeur de verrou est égale à 0 (section critique disponible). A la sortie de la boucle, la valeur du verrou est remise à 1 pour indiquer que le processus va entrer dans sa section critique. En ce qui concerne, la définition de fin de la section critique, on utilise une instruction qui remet verrou à 0 vu que la section critique est redevenue disponible :

```
Tant que (verrou!=0) Faire
sleep
Fintantque
Verrou:=1
<section critique>
Verrou:=0
```

Les solutions avec attente active sont gourmandes en temps CPU. En effet, même si un processus n'avance pas dans l'exécution de ses instructions, il consomme du temps CPU dans l'attente de la disponibilité de la section critique (exécution de la boucle tant que la section critique n'est pas disponible).

3. Solutions avec blocage

Le principe de solutions avec blocage consiste à faire bloquer un processus qui ne peut pas entrer dans sa section critique. Le processus en question passe ainsi de l'état Exécution vers l'état Attente (Diagramme d'états des

processus). Lorsque la section critique redevient disponible, le processus qui était dans la section critique envoie un signal à un des processus bloqués pour qu'il passe de l'état Attente vers l'état Prêt. Ainsi, tant que la section critique n'est pas disponible, les processus concernés par l'attente n'utilisent pas le temps CPU puisqu'ils sont bloqués.
Le début d'une section critique est défini par une condition qui vérifie la disponibilité ou non de la section critique. Si la section critique n'est pas disponible, le processus se bloque. A la fin de la section critique, le processus donne la main à un des processus en attente de la section critique. Il s'agit de réveiller le processus suivant par l'envoie d'un signal. Ce processus passe ainsi de l'état Attente vers l'état Prêt :

```
...
Est-ce que je peux entrer en SC ?
Si NON alors je me bloque ...
<section critique>
Est-ce qu'un processus est bloqué ?
Si OUI alors le réveiller (ou Ordonner un réveil)
```

IV. Conclusion

Dans ce chapitre, nous avons traité du problème de synchronisation de processus nécessaire en multiprogrammation et dans le cas de partage de ressources. Plusieurs solutions sont proposées pour la synchronisation de processus (verrouillage, attente active, blocage). Dans le cadre de ce cours, nous avons juste donné un aperçu général de quelques solutions de synchronisation. Ces solutions existent sous forme de plusieurs algorithmes de synchronisation : le modèle producteur consommateur, la synchronisation par sémaphores, le dîner des philosophes mangeurs de Spaghetti, les solutions matérielles. Toutes ces solutions peuvent être reprises en détails dans le cadre de cours plus avancés en système d'exploitation.

Chapitre 5 Gestion de la mémoire

I. Introduction

La mémoire est une partie de l'unité centrale dans laquelle sont temporairement chargées les instructions et les données des programmes en cours d'exécution. Outre sa capacité, une gestion optimale de la mémoire contribuerait inévitablement à améliorer l'efficacité du système d'exploitation en question et améliorer ainsi sa performance. Le rôle de la gestion de la mémoire ne se résume pas en la simple allocation de la mémoire à des processus lorsqu'ils en ont besoin.

II. Rôle de la gestion de la mémoire

La mémoire centrale est un espace qui permet de mémoriser temporairement les instructions et les données intermédiaires ou les résultats de calculs du ou des programmes en cours d'exécution. La gestion mémoire la plus simple consiste à avoir un seul processus en mémoire à un instant donné, c'est le principe de la monoprogrammation. Le système d'exploitation charge un programme donné en mémoire, lorsque le programme se termine, le système d'exploitation charge un autre programme. La multiprogrammation a été présentée dans le premier chapitre comme un moyen pour améliorer l'efficacité de l'utilisation du processeur. En effet, l'exécution d'un processus est une alternance entre de calculs effectués par le processeur et des opérations d'entrées sorties effectuées par les périphériques. Avec la multiprogrammation, la mémoire contient plusieurs processus, lorsqu'un processus demande une entrée sortie, le processeur est alloué à un autre programme en mémoire. Avec la multiprogrammation la gestion de la mémoire doit assurer les fonctions suivantes :

- Définir un espace d'adressage indépendant pour chaque processus.
- Protéger les espaces mémoires utilisés par les processus (empêcher les accès interdits).
- Optimiser le partage de la mémoire. Allouer de la mémoire aux processus en évitant au maximum le gaspillage.
- Récupérer la mémoire libérée par la terminaison d'un processus.
- Connaître les parties libres de la mémoire physique.

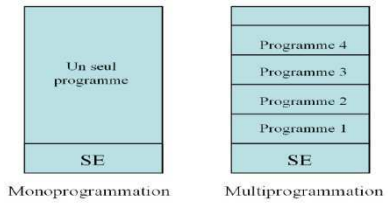


Figure 5 La multiprogrammation

III. Partitions fixes

Avec la stratégie de partitions fixes, la mémoire est divisée en des partitions de tailles égales ou différentes. Une partition peut accueillir un seul processus. Il existe deux alternatives d'allocation des partitions aux processus. La première appelée stratégie de partitions fixes à files multiples consiste à implanter une file d'attente par partition :

- L'allocation de la mémoire physique est obtenue en associant une file d'attente par partition.
- Les processus d'une file doivent être de taille inférieure ou égale à la taille de la partition correspondante.
- Un processus attend dans une file la disponibilité de la partition qui peut l'accueillir.

L'utilisation de files multiples pose problème lorsque les files des grandes partitions sont vides et les autres sont pleines. Ainsi, une grande partition peut être libre avec une file vide ne peut pas être utilisée par les processus placés dans les autres files des partitions plus petites : gaspillage de la mémoire. Une deuxième alternative appelée stratégie de partitions fixes à file unique consiste à implanter une seule file d'attente pour toutes les partitions. Ainsi, dès qu'une partition est libre, on y place un processus de la file de taille inférieure ou égale à la taille de la partition. Un exemple de machines qui utilisaient cette technique est la machine OS/360 d'IBM [EF08].

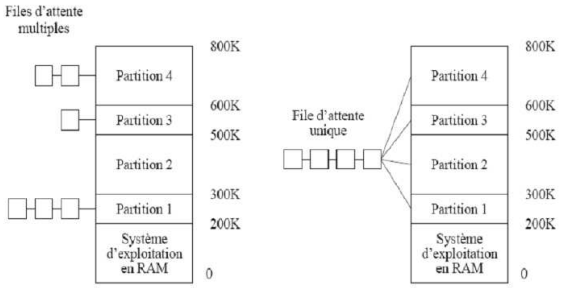


Figure 6 Stratégie de partition fixe avec files multiples et file variable

IV. Partitions dynamiques ou variables

L'organisation de la mémoire en partitions fixes est une méthode facile à mettre en œuvre. Néanmoins, elle engendre un gaspillage de la mémoire dû au fait que la taille d'une partition peut être supérieure à la taille du processus qui lui est affecté. Une partition ne peut contenir qu'un seul processus, l'espace mémoire qui reste dans chaque partition est inutilisable. La stratégie de partitions variables consiste à allouer à chaque processus une zone de la mémoire de taille exactement égale à sa taille. On parle ainsi d'allocation à la demande :

- La mémoire est découpée en partitions de taille variable, c'est-à-dire adaptables à la taille des programmes.
- La mémoire est formée d'un ensemble de zones libres et de zones occupées ou allouées.
- Le gestionnaire de la mémoire doit maintenir une liste de zones libres en mémoire. Cette liste est mise-à-jour à chaque allocation ou désallocation d'un processus.
- Une zone libre est caractérisée par son adresse d'implantation et sa taille.
- L'allocation des zones mémoires aux processus est réalisée comme suit :
  - Les programmes déclarent la taille dont ils ont besoin au chargement.
  - Le gestionnaire de la mémoire cherche une zone libre de taille suffisante pour contenir le processus en attente (taille(zone libre) ¸ taille(processus)). Deux stratégies de recherche de zones libres peuvent être utilisées : la première stratégie First Fit consiste à placer le processus en attente dans la première zone qui convient. La deuxième stratégie Best Fit consiste à choisir celle générant le moins de perte.

La stratégie de partition fixe peut également engendrer un gaspillage de la mémoire. En effet, les allocations et désallocations successives des programmes créent des trous qui sont des zones libres de taille insuffisante. Ces zones libres ne sont pas utilisables puisqu'elles ne sont pas contiguës. Une solution pour ce problème consiste à réaliser un compactage de la mémoire.

Le compactage consiste à déplacer les programmes en mémoire centrale de manière à ne créer qu'une seule et unique zone libre. C'est une opération très coûteuse en temps CPU. Elle est donc effectuée le moins souvent possible.

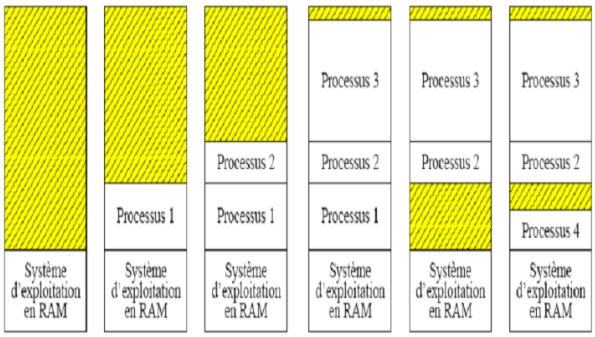


Figure 7 Stratégie de partition dynamique

V. Gestion avec pagination ou segmentation : la mémoire virtuelle

La taille des programmes exécutés peut être très importante pour tenir entièrement en mémoire. Le principe de la mémoire virtuelle consiste à conserver en mémoire que les parties utiles à l'avancement immédiat des processus. Le reste des parties est conservé sur le disque. La mémoire virtuelle fait appel au mécanisme de segmentation.

Au lieu d'être entièrement chargé à des adresses contiguës comme c'est le cas des approches de partitions fixes ou dynamiques, un processus peut être chargé dans des pages ou des segments non contigus.
Le but de la mémoire virtuelle est de pouvoir offrir à chaque processus une mémoire plus grande que celle disponible. Le disque est ainsi utilisé comme mémoire supplémentaire. Ainsi, les pages utiles d'un processus sont chargés en mémoire physique, les autres sont placées sur le disque.
Lorsque la mémoire est pleine, certaines pages migrent vers le disque et on charge de nouvelles pages en mémoire centrale. Les pages sont stockées sur le disque dans une partition appelée swap (ou zone d'échange). La mise en place d'une mémoire virtuelle est basée sur le principe de la pagination qu'on peut définir comme suit :

- Un processus est divisé en des parties de taille fixe appelées pages.
- La mémoire centrale est également découpée en pages.
- Le système ne charge en mémoire physique que les pages de processus utiles.
- La mémoire virtuelle est également "découpée" en pages. Une table de pages est utilisée pour faire correspondre à chaque page de la mémoire virtuelle la page correspondante dans la mémoire physique (les données étant réellement stockées dans la mémoire physique).
- Le calcul des adresses physiques se fait à l'aide d'une unité qui se trouve entre le CPU et la mémoire centrale, la MMU (Memory Mangement Unit ou unité de gestion de mémoire). La MMU utilise la table des pages pour traduire les adresses virtuelles en adresses physiques.

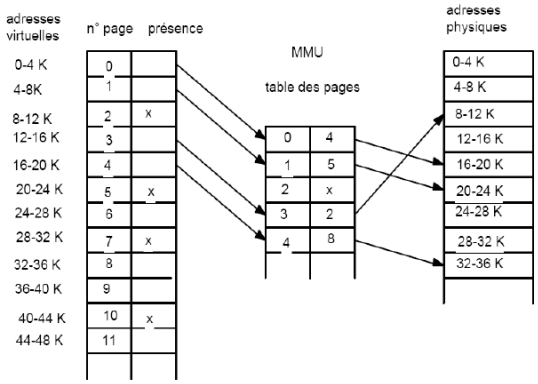


Figure 8 Gestion avec pagination ou segmentation : la mémoire virtuelle

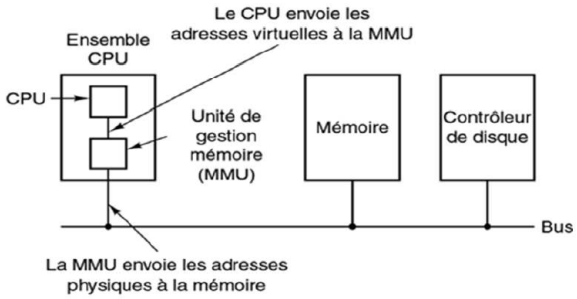


Figure 9 Unité de gestion de mémoire (MMU)

Les pages chargées dans la mémoire physiques sont remplacées par de nouvelles pages du swap en fonction des besoins d'exécution. Ceci nécessite une gestion des pages et un algorithme de remplacement de pages. Le système choisit quelle page doit être enlevée pour faire de la place à la nouvelle page. Il est préférable de ne pas choisir une page souvent utilisée puisqu'elle risque d'être aussitôt rechargée. Nous définissons dans ce qui suit quelques algorithmes de remplacement de page :

- Remplacement de page optimal : il s'agit de remplacer la page qui sera utilisée après toutes les autres. Cet algorithme est pratiquement impossible à appliquer.
- Algorithme premier arrivé, premier sorti : First In First Out (FIFO) : on maintient une liste de pages dans l'ordre de leur chargement en mémoire. La page au début de la liste (la plus ancienne) est remplacée. Cet algorithme est simple à mettre en œuvre mais présente le désavantage qui consiste au fait que les vieilles pages peuvent aussi être celles qui sont le plus utilisées.
- Algorithme de la page la moins récemment utilisée : Last Recently Used (LRU) : on suppose que les pages qui ont été utilisées récemment le seront à nouveau prochainement. On enlève ainsi les pages qui n'ont pas été utilisées depuis longtemps. Un compteur C est incrémenté après chaque instruction. La page avec la plus petite valeur de compteur est celle qui sera remplacée. Les pages peuvent ne pas garder le même comportement (le nombre d'utilisation peut changer), le compteur C est ainsi remis à 0 périodiquement.

VI. Conclusion

Nous avons présenté dans ce chapitre le rôle de la gestion de la mémoire ainsi que les principales stratégies de gestion de mémoire définies dans la littérature. Pour pouvoir appliquer le principe de la multiprogrammation, un système de gestion de mémoire est nécessaire. Ce système a pour objectif de partager de façon optimale (avec un minimum de gaspillage) la mémoire entre les processus. Elle doit ainsi gérer les zones libres, les zones occupées tout en assurant une protection et une séparation des zones de chaque processus. Les stratégies présentées dans ce chapitre peuvent être regroupées en deux types : les stratégies qui nécessitent un chargement total des processus dans un espace d'adressage contigu (partitions fixes et partitions variables). Le deuxième type correspond aux stratégies des mémoires virtuelles où les processus peuvent être partiellement chargés en mémoire dans des espaces d'adressages séparées (les pages) et pas nécessairement contigus.

Chapitre 6 Système de gestion des fichiers

I. Introduction

Problématique : Le volume des données traitées par les applications informatiques atteignant plusieurs méga et giga octets, ces données ne peuvent pas être stockées dans la mémoire centrale. On souhaite également disposer d'un stockage à long terme qui ne disparaisse pas lorsqu'on éteint la machine.
Solution : Le principe consiste à stocker ces données dans des mémoires secondaires sous forme de fichiers, c'est-à-dire de suites de blocs (la plus petite unité que le périphérique de stockage est capable de gérer).
Le contenu de ces blocs, simple suite de chiffres binaires, peut être interprété selon le format de fichier comme des caractères, des nombres entiers ou flottants, des codes d'opérations machines, des adresses mémoires, etc.
L'échange entre les deux types de mémoires se fait ensuite par transfert de blocs
L'objectif du système de gestion de fichiers : est de permettre l'accès au contenu du fichier (l'ouverture et la fermeture du fichier, sa création, lecture et écriture dans le fichier, sa recopie à un second emplacement ou sa suppression) à partir de son chemin d'accès, formé d'un nom précédé d'une liste de répertoires imbriqués.

II. Le concept de fichier

Un fichier est un ensemble d'informations structurées regroupées en vue de leur conservation (notion de persistance) et de leur réutilisation dans un système informatique.

- Un fichier logique est le regroupement de tous les enregistrements décrivant le même type d'entité.



Figure 10 Structure d'un fichier logique

- Un fichier physique est un ensemble de blocs physiques composé chacun par 1 à *n secteurs.* Le secteur étant l'unité d'allocation du disque



Figure 11 Structure d'un bloc

Exemple de fichier logique:

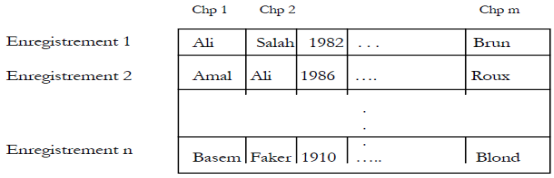


Figure 12 Exemple de fichier logique

Le système d'exploitation établit la correspondance entre le fichier et le système binaire utilisé lors du stockage de manière transparente pour les utilisateurs. Dans un fichier on peut écrire du texte, des images, des calculs, des programmes….
Les fichiers sont généralement créés par les utilisateurs. Toutefois certains fichiers sont générés par le système ou certains outils comme les compilateurs.
Afin de différencier les fichiers entre eux, chaque fichier a un ensemble d'attributs qui le décrivent. Parmi ceux-ci on retrouve : le nom, l'extension, la date et l'heure de sa création ou de sa dernière modification, la taille, la protection. Certains attributs sont indiqués par l'utilisateur, d'autres sont complétés par le système d'exploitation.

Un système de fichiers est une collection de fichiers.
Un système de fichiers est représenté par une structure de données arborescente. Les fichiers sont regroupées dans des répertoires. Un répertoire peut contenir soit des fichiers, soit d'autres répertoires.

Le répertoire peut être considéré comme une table sur le support permettant de référencer tous les fichiers existants avec leurs noms et leurs caractéristiques principales qui peuvent varier d'un système d'exploitation à un autre. Parmi ces caractéristiques, on peut citer le nom, le type, la taille, le propriétaire, la protection, la date de création et la localisation sur disque (ou l'adresse du premier enregistrement).

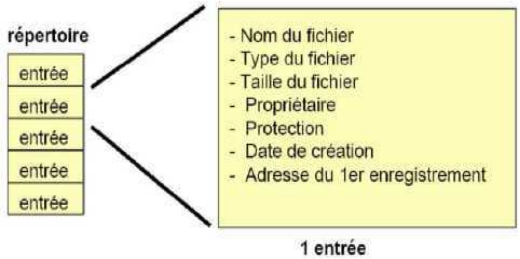


Figure 13 Structure d'un répertoire

Chaque fichier possède un chemin unique formé de la liste des répertoires qu'il faut traverser à partir du répertoire racine jusqu'au fichier. C'est ce qu'on appelle le chemin absolu. Le chemin relatif permet d'identifier l'emplacement d'un fichier à partir du répertoire courant.

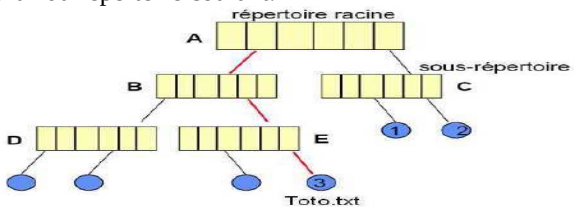


Figure 14 Organisation de fichiers et répertoires : Structure arborescente

Dans la figure, le fichier toto.txt possède comme chemin absolu A/B/E/ et comme chemin relatif au répertoire B, B/E/. Dans l'identification d'un chemin, les répertoires sont séparés par un caractère qui dépend du système d'exploitation : / pour UNIX, *n pour Dos et Winxx, : pour* MacOS.

Un système de gestion de fichiers (SGF ou file system en anlgais) est un ensemble de méthodes et de structures de données permettant de manipuler les données (fichiers et répertoires) stockées sur des mémoires secondaires. Les données sont regroupées par le SGF dans des fichiers localisés à partir d'un chemin d'accès.

III. Les fonctionnalités d'un SGF

Il a comme fonctionnalités :

- **Conservation permanente des fichiers.**
- **Manipulation des fichiers :** le SGF fournie un certain nombre d'opérations pour la manipulation des fichiers et des répertoires (création, insertion, suppression, modification).
- **Allocation de zones mémoires :** le SGF alloue à chaque fichier un nombre variable de blocs (suite d'octets de taille fixe).
- **Localisation des fichiers :** chaque fichier est décrit par un ensemble d'informations qui permettent d'identifier son emplacement (nom, adresse, etc.).
- **Partage des fichiers :** assurer une utilisation d'un même fichier / répertoire / disque par des utilisateurs différents.
- **Protection des fichiers :** le SGF doit assurer la sécurité et la confidentialité des données surtout en cas de partage. Des droits d'accès sont ainsi associés à chaque fichier pour assurer une protection contre les accès interdits.

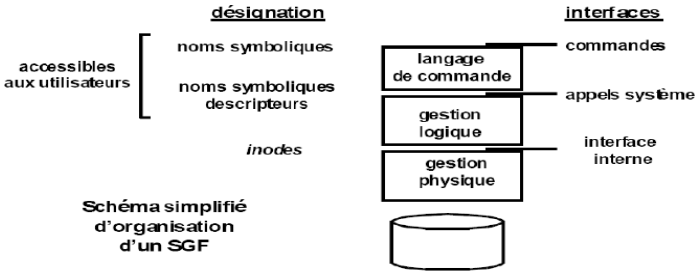


Figure 15 Organisation d'un SGF

L'utilisateur possède une vue abstraite de l'organisation des fichiers qui sont désignés par des noms symboliques. Chaque système d'exploitation possède un système d'identification interne des fichiers. Le système d'exploitation UNIX, par exemple, identifie ses fichiers par ce qu'on appelle un numéro d'inode ou i-number. La gestion logique concerne l'organisation des fichiers et des répertoires ainsi que toutes les fonctions qui vont avec comme la gestion des droits d'accès, le partage et la protection. La gestion physique concerne l'allocation de l'espace disque (différentes stratégies d'allocation de l'espace disque sont définies en détail dans la section suivante).

IV. Allocation de l'espace disque

Un fichier physique est constitué d'un ensemble de blocs physiques. La question qui se pose c'est comment allouer des blocs physiques pour les fichiers. Différents points doivent être pris en considération pour l'allocation de l'espace disque : l'optimisation du temps d'accès, le coût de stockage et l'évolution de la taille des fichiers (la fragmentation).

1. Allocation contiguë

Comme son nom l'indique, la méthode d'allocation contiguë consiste à stocker chaque fichier dans une suite de blocs contigus. Chaque fichier est ainsi décrit par l'adresse disque du premier bloc et sa taille. Les adresses des différents blocs d'un fichier sont obtenues par simple addition.

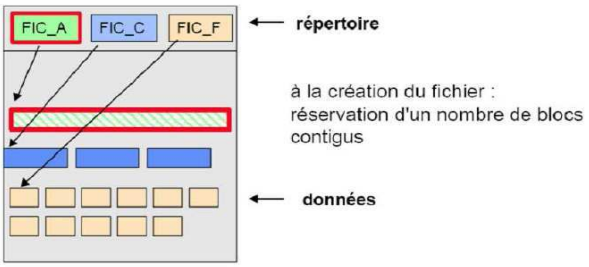
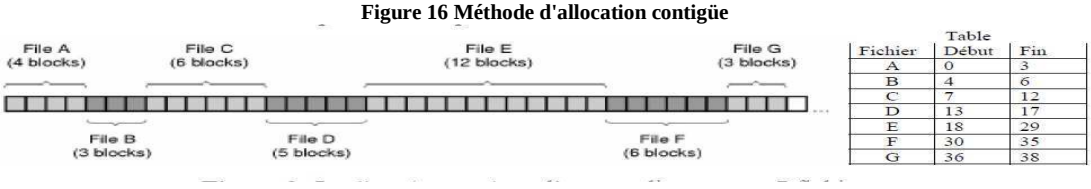


Figure 16 Méthode d'allocation contigüe



Table

Fichier	Début	Fin
A	0	3
B	4	6
C	7	12
D	13	17
E	18	29
F	30	35
G	36	38

Ajout d'un enregistrement : l'enregistrement est ajouté à la suite du dernier bloc du fichier s'il y a suffisamment d'espace sinon il serait nécessaire de réaliser un déplacement du fichier dans un espace plus grand (réorganisation).
Suppression d'un fichier : on parle de suppression logique. Un compteur est associé à chaque entrée fichier dans le répertoire correspondant. Ce compteur indique le nombre d'enregistrements du fichier. Dans le cas de la suppression, ce compteur est remis à zéro.
Avantage : rapidité de l'accès (les blocs étant contigus, on limite les déplacements de la tête le lecture/écriture, coûteux en temps).
Inconvénients :

- Le dernier bloc a toutes chances d'être sous-utilisé et ainsi, **on gaspille de la place.**
- Il est difficile de prévoir la taille qu'il faut réserver au fichier
- La perte d'espace sur le disque
- Problème de **fragmentation externe**

2. Allocation chaînée (non contigüe)

La méthode d'allocation chaînée est proposée pour résoudre les problèmes de la méthode d'allocation contiguë. En effet, avec cette méthode, chaque fichier est une liste chaînée de blocs de disques. Ces blocs peuvent être dans n'importe quel emplacement sur le disque. Le répertoire contient un pointeur sur le premier et le dernier bloc du fichier. Chaque bloc, excepté le dernier, contient un pointeur vers le bloc suivant. Le dernier bloc d'un fichier contient un pointeur vers le premier emplacement libre.

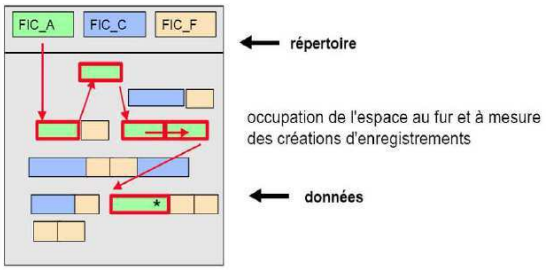
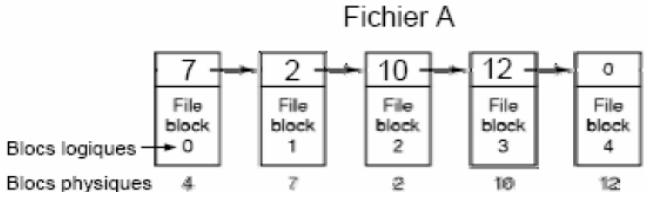


Figure 17 Méthode d'allocation chaînée



Création d'un fichier : La méthode d'allocation chaînée est proposée pour résoudre les problèmes de la méthode d'allocation contiguë. En effet, avec cette méthode, chaque fichier est une liste chaînée de blocs de disques. Ces blocs peuvent être dans n'importe quel emplacement sur le disque.
Le répertoire contient un pointeur sur le premier et le dernier bloc du fichier. Chaque bloc, excepté le dernier, contient un pointeur vers le bloc suivant. Le dernier bloc d'un fichier contient un pointeur vers le premier emplacement libre.
Ajout d'un enregistrement : l'ajout d'un enregistrement consiste à rechercher des blocs disponibles pour contenir l'enregistrement. A chaque fois qu'on alloue un bloc on le chaîne au dernier bloc trouvé.
Suppression d'un fichier : la suppression suit le même principe de la suppression dans le cas d'allocation contiguë.
Avantage :

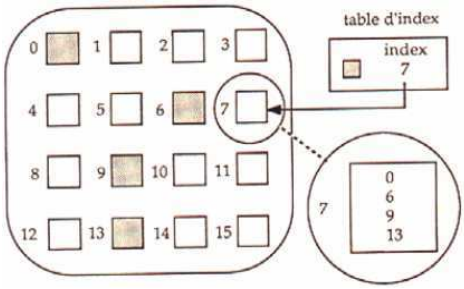
- Elimination du problème de fragmentation externe
- Plus simple d'étendre un fichier (ajout d'un enregistrement) que dans le cas de la méthode contiguë. Également, grâce à l'allocation par bloc individuel, tout bloc libre peut être utilisé

Inconvénients :

- L'accès au fichier est totalement séquentiel, on doit toujours commencer le parcours du fichier à partir du début.
- La perte d'un chaînage entraîne la perte de tout le reste du fichier. Pire encore, il suffit qu'une valeur soit modifiée dans un pointeur pour qu'on se retrouve dans une autre zone de la mémoire.

3. Allocation non contigüe indexée

Définition : Tous les inconvénients de l'allocation chaînée peuvent être résolus d'une manière simple : il suffit de retirer les pointeurs des blocs et de les placer dans une structure de données gardée en mémoire centrale, ainsi, les informations sur les numéros de blocs peuvent être obtenue à tout moment.



La i^{e} entrée du bloc d'index pointe sur le i^{e} bloc de données du fichier.
La plus part des systèmes actuels appliquent ce mode. MS-DOS utilise la **FAT (File Allocation Table) pour** y conserver les chaînages entre les blocs. Windows NT utilise la **MFT (Master File Table) associé au** système **NTFS (New Technology File System)**.UNIX, GNU/Linux utilisent le **I-Node (Index node).**
Avantages

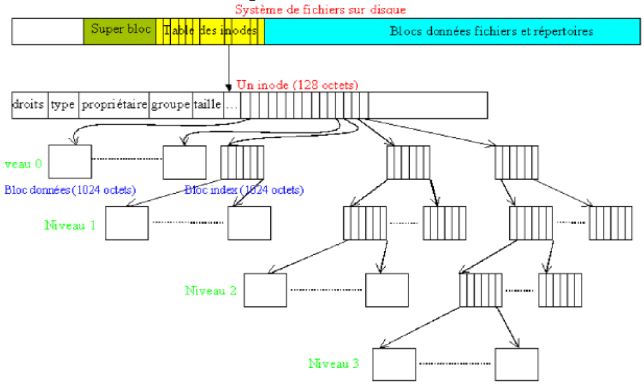
- Implémentation efficace de l'accès direct
- Table des i-nodes de taille proportionnelle au nombre de fichiers (Cf. FAT : taille du disque)

Inconvénients

- Problème de la taille des index

V. Le système de gestion de fichiers de UNIX

Le système de gestion de fichiers d'UNIX est appelé UFS pour Unix File System.
On distingue trois types de fichiers : les fichiers standards ou ordinaires (ex. texte), les répertoires et les fichiers spéciaux (ex. périphériques).
Les fichiers sont identifiés par un numéro unique le numéro d'inode. Un répertoire n'est qu'un fichier, identifié aussi par un inode, contenant une liste d'inode représentant chacun un fichier.



Chaque entrée de la table des inodes (ou bloc d'index) contient les informations sur les fichiers excepté le nom.
Chaque inode contient un ensemble de pointeurs :

- Les premiers sont des pointeurs directs qui pointent vers les blocs de données de fichiers
- Les derniers sont des pointeurs indirects qui pointent vers les blocs d'index. Ces derniers contiennent à leur tour une nouvelle série de pointeurs directs et indirects.

Un répertoire sur UNIX est un fichier qui contient une liste de couples (nom de fichier, numéro inode).
Lors de la suppression d'un fichier, c'est uniquement l'entrée correspondante du fichier dans le répertoire qui sera supprimée. Les données du fichiers restent physiquement stockées sur le disque tant que rien n'a été écrit à sa place.

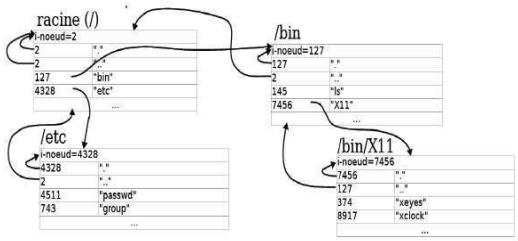


Figure 18 Organisation des répertoires d'UNIX

Bibliographies

- Cours Systèmes d'exploitation I d'Afef Bahri. Ecole supérieure de commerce électronique, Manouba.
- Cours Les systèmes d'exploitation d'Imen Sghaïeri. Institut supérieur des études technologiques de Nabeul.
- Cours Systèmes d'exploitation des ordinateurs de Leila Baccouche et Mona Laroussi. Université Virtuelle, Tunis.
- Cours Systèmes d'exploitation de Remi Bonidal. Université de Lorraine, Nancy.
- Cours Systèmes d'exploitation d'Omar Megzari. Faculté des Sciences, Rabat.
-